

機器學習 HW4
黃得晉 113352019 金融碩

以下先做資料處理 把 P hat, R_ls, R_lo, R_b

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

data = pd.read_excel("/Users/huangdejin/Desktop/R\Python\Code\matlab/金融科技/final_result.xlsx")

features = [
    "本益比-TEJ", "股價淨值比-TEJ", "股價營收比-TEJ", "現金股利率",
    "週轉率%_月", "高低價差%", "成交量(百萬股)_月", "成交值比重%"
]
data["return"] = data["報酬率%_月"] / 100
data["y"] = (data["return"] > 0).astype(int) # Target variable

results = []
gamma = 0.5
lambda_ridge = 0.5
scaler = StandardScaler()

data["年月"] = data["年月"].astype(int)
times = sorted(data["年月"].unique())[1:]

for t in times:

    train_data = data[data["年月"] == t - 1]

    test_data = data[data["年月"] == t]

    next_data = data[data["年月"] == t + 1]

    if train_data.empty or test_data.empty or next_data.empty:
        continue

    common_stocks = set(train_data["證券代碼"]).intersection(
        set(test_data["證券代碼"]), set(next_data["證券代碼"])
    )
    if not common_stocks:
        continue
```

```

train_data = train_data[train_data["證券代碼"].isin(common_stocks)]
test_data = test_data[test_data["證券代碼"].isin(common_stocks)]
next_data = next_data[next_data["證券代碼"].isin(common_stocks)]

train_data = train_data.sort_values("證券代碼")
test_data = test_data.sort_values("證券代碼")
next_data = next_data.sort_values("證券代碼")

X_train = train_data[features].values
y_train = train_data["y"].values
X_test = test_data[features].values
R_next = next_data["return"].values

if np.any(np.isnan(X_train)) or np.any(np.isnan(X_test)) or np.any(np.isnan(R_next)):
    continue # Skip periods with missing values

if len(np.unique(y_train)) < 2:
    print(f"Skipping t={t}: y_train has only one class ({np.unique(y_train)})")
    continue

# Standardize features
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LogisticRegression(penalty="l2", C=1/lambda_ridge, solver="lbfgs")
model.fit(X_train, y_train)

# Predict probabilities
p_hat = model.predict_proba(X_test)[:, 1]

# Verify shapes
if len(p_hat) != len(R_next):
    print(f"Shape mismatch at t={t}: p_hat={len(p_hat)}, R_next={len(R_next)}")
    continue

```

```

# Calculate quantities
prop_positive = np.mean(y_train)
prop_p_hat = np.mean(p_hat > gamma)
quantiles = np.percentile(p_hat, [0, 5, 25, 50, 75, 95, 100])
R_ls = np.mean(np.sign(p_hat - gamma) * R_next)
R_lo = np.mean((p_hat > gamma).astype(int) * R_next)
R_b = np.mean(R_next)

results.append({
    "time": t,
    "prop_positive": prop_positive,
    "prop_p_hat": prop_p_hat,
    "quantiles": quantiles,
    "R_ls": R_ls,
    "R_lo": R_lo,
    "R_b": R_b
})

```

✓ 3.1s

Q1

```

# Problem 1: Mean-volatility ratio
R_ls_series = np.array([r["R_ls"] for r in results])
R_lo_series = np.array([r["R_lo"] for r in results])
R_b_series = np.array([r["R_b"] for r in results])
mv_ls = np.mean(R_ls_series) / np.std(R_ls_series) if np.std(R_ls_series) != 0 else np.nan
mv_lo = np.mean(R_lo_series) / np.std(R_lo_series) if np.std(R_lo_series) != 0 else np.nan
mv_b = np.mean(R_b_series) / np.std(R_b_series) if np.std(R_b_series) != 0 else np.nan
print(f"MV Ratios: LS={mv_ls:.4f}, LO={mv_lo:.4f}, B={mv_b:.4f}")

```

✓ 0.0s

MV Ratios: LS=0.0787, LO=0.2075, B=0.2134

根據輸出的結果 用 LS 或 LO 算出來的結果都小於 benchmark 的結果

Q2

```
# Problem 2: Relationship between positive return proportion and predicted probabilities
prop_positive = np.array([r["prop_positive"] for r in results])
prop_p_hat = np.array([r["prop_p_hat"] for r in results])
prop_future = np.array([
    data[data["年月"] == r["time"] + 1]["y"].mean()
    if r["time"] + 1 in data["年月"].values else np.nan
    for r in results
])
corr_p_hat_future = np.corrcoef(
    prop_p_hat[~np.isnan(prop_future)], prop_future[~np.isnan(prop_future)]
)[0, 1] if np.sum(~np.isnan(prop_future)) > 1 else np.nan
print(f"Prop Positive Std: {np.std(prop_positive):.4f}")
print(f"Correlation (Prop_p_hat, Prop_future): {corr_p_hat_future:.4f}")
✓ 0.0s
```

```
Prop Positive Std: 0.2269
Correlation (Prop_p_hat, Prop_future): -0.0070
```

根據結果表示 正收益比例會隨時間增加而增加，然後因為相關性算出來是接近零，所以表示模型無法預測下一期的收益

Q3

```

# Problem 3: Lower gamma to 0.4
results_gamma_04 = []
gamma = 0.4
for t in times:

    train_data = data[data["年月"] == t - 1]
    test_data = data[data["年月"] == t]
    next_data = data[data["年月"] == t + 1]

    if train_data.empty or test_data.empty or next_data.empty:
        continue

    common_stocks = set(train_data["證券代碼"]).intersection(
        set(test_data["證券代碼"]), set(next_data["證券代碼"])
    )
    if not common_stocks:
        continue

    train_data = train_data[train_data["證券代碼"].isin(common_stocks)]
    test_data = test_data[test_data["證券代碼"].isin(common_stocks)]
    next_data = next_data[next_data["證券代碼"].isin(common_stocks)]

    train_data = train_data.sort_values("證券代碼")
    test_data = test_data.sort_values("證券代碼")
    next_data = next_data.sort_values("證券代碼")

    X_train = train_data[features].values
    y_train = train_data["y"].values
    X_test = test_data[features].values
    R_next = next_data["return"].values

    if np.any(np.isnan(X_train)) or np.any(np.isnan(X_test)) or np.any(np.isnan(R_next)):
        continue

    if len(np.unique(y_train)) < 2:
        continue

```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(penalty="l2", C=1/lambda_ridge, solver="lbfgs")
model.fit(X_train, y_train)

p_hat = model.predict_proba(X_test)[: , 1]

if len(p_hat) != len(R_next):
    continue

prop_positive = np.mean(y_train)
prop_p_hat = np.mean(p_hat > gamma)
quantiles = np.percentile(p_hat, [0, 5, 25, 50, 75, 95, 100])
R_ls = np.mean(np.sign(p_hat - gamma) * R_next)
R_lo = np.mean((p_hat > gamma).astype(int) * R_next)
R_b = np.mean(R_next)

results_gamma_04.append({
    "time": t,
    "prop_positive": prop_positive,
    "prop_p_hat": prop_p_hat,
    "quantiles": quantiles,
    "R_ls": R_ls,
    "R_lo": R_lo,
    "R_b": R_b
})

mv_ls_04 = np.mean([r["R_ls"] for r in results_gamma_04]) / np.std([r["R_ls"] for r in results_gamma_04]) if np.std([r["R_ls"] for r in results_gamma_04]) != 0 else np.nan
mv_lo_04 = np.mean([r["R_lo"] for r in results_gamma_04]) / np.std([r["R_lo"] for r in results_gamma_04]) if np.std([r["R_lo"] for r in results_gamma_04]) != 0 else np.nan
print(f"Question 3 (gamma=0.4): MV_LS={mv_ls_04:.4f}, MV_LO={mv_lo_04:.4f}")

✓ 0.5s
Question 3 (gamma=0.4): MV_LS=0.1085, MV_LO=0.2047
```

當我將 `gamma` 調整成 0.4 之後用 LS 或 LO 算出來的結果依舊都是小於 benchmark 的結果

Q4

```

# Problem 4: Increase lambda_ridge to 5
results_lambda_5 = []
gamma = 0.5
lambda_ridge = 5
for t in times:
    # (Repeat the loop with lambda_ridge=5, same structure as above)
    train_data = data[data["年月"] == t - 1]
    test_data = data[data["年月"] == t]
    next_data = data[data["年月"] == t + 1]

    if train_data.empty or test_data.empty or next_data.empty:
        continue

    common_stocks = set(train_data["證券代碼"]).intersection(
        set(test_data["證券代碼"]), set(next_data["證券代碼"])
    )
    if not common_stocks:
        continue

    train_data = train_data[train_data["證券代碼"].isin(common_stocks)]
    test_data = test_data[test_data["證券代碼"].isin(common_stocks)]
    next_data = next_data[next_data["證券代碼"].isin(common_stocks)]

    train_data = train_data.sort_values("證券代碼")
    test_data = test_data.sort_values("證券代碼")
    next_data = next_data.sort_values("證券代碼")

    X_train = train_data[features].values
    y_train = train_data["y"].values
    X_test = test_data[features].values
    R_next = next_data["return"].values

    if np.any(np.isnan(X_train)) or np.any(np.isnan(X_test)) or np.any(np.isnan(R_next)):
        continue

    if len(np.unique(y_train)) < 2:
        continue

```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(penalty="l2", C=1/lambda_ridge, solver="lbfgs")
model.fit(X_train, y_train)

p_hat = model.predict_proba(X_test)[:, 1]

if len(p_hat) != len(R_next):
    continue

prop_positive = np.mean(y_train)
prop_p_hat = np.mean(p_hat > gamma)
quantiles = np.percentile(p_hat, [0, 5, 25, 50, 75, 95, 100])
R_ls = np.mean(np.sign(p_hat - gamma) * R_next)
R_lo = np.mean((p_hat > gamma).astype(int) * R_next)
R_b = np.mean(R_next)

results_lambda_5.append({
    "time": t,
    "prop_positive": prop_positive,
    "prop_p_hat": prop_p_hat,
    "quantiles": quantiles,
    "R_ls": R_ls,
    "R_lo": R_lo,
    "R_b": R_b
})

mv_ls_5 = np.mean([r["R_ls"] for r in results_lambda_5]) / np.std([r["R_ls"] for r in results_lambda_5]) if np.std([r["R_ls"] for r in results_lambda_5]) != 0 else np.nan
mv_lo_5 = np.mean([r["R_lo"] for r in results_lambda_5]) / np.std([r["R_lo"] for r in results_lambda_5]) if np.std([r["R_lo"] for r in results_lambda_5]) != 0 else np.nan
print(f"Question 4 (lambda=5): MV_LS={mv_ls_5:.4f}, MV_LO={mv_lo_5:.4f}")
✓ 0.4s

Question 4 (lambda=5): MV_LS=0.0623, MV_LO=0.1953
```

第一部分用 LS 或 LO 算出來的結果依舊都是小於 benchmark 的結果

```
prop_positive_lambda_5 = np.array([r["prop_positive"] for r in results_lambda_5])
prop_p_hat_lambda_5 = np.array([r["prop_p_hat"] for r in results_lambda_5])
prop_future_lambda_5 = np.array([
    data[data["年月"] == r["time"] + 1]["y"].mean()
    if r["time"] + 1 in data["年月"].values else np.nan
    for r in results_lambda_5
])
corr_p_hat_future_lambda_5 = np.corrcoef(
    prop_p_hat_lambda_5[~np.isnan(prop_future_lambda_5)], prop_future_lambda_5[~np.isnan(prop_future_lambda_5)]
)[0, 1] if np.sum(~np.isnan(prop_future_lambda_5)) > 1 else np.nan
print(f"Prop Positive Std: {np.std(prop_positive_lambda_5):.4f}")
print(f"Correlation (Prop_p_hat, Prop_future): {corr_p_hat_future_lambda_5:.4f}")
✓ 0.0s

Prop Positive Std: 0.2269
Correlation (Prop_p_hat, Prop_future): -0.0196
```

第二部分 根據結果表示 正收益比例會隨時間增加而增加，然後因為相關性算出來是負的，所以表示模型無法預測下一期的收益


```

# Problem 4: Increase lambda_ridge to 5
results_lambda_5_gamma_04 = []
gamma = 0.4
lambda_ridge = 5
for t in times:
    # (Repeat the loop with lambda_ridge=5, same structure as above)
    train_data = data[data["年月"] == t - 1]
    test_data = data[data["年月"] == t]
    next_data = data[data["年月"] == t + 1]

    if train_data.empty or test_data.empty or next_data.empty:
        continue

    common_stocks = set(train_data["證券代碼"]).intersection(
        set(test_data["證券代碼"]), set(next_data["證券代碼"])
    )
    if not common_stocks:
        continue

    train_data = train_data[train_data["證券代碼"].isin(common_stocks)]
    test_data = test_data[test_data["證券代碼"].isin(common_stocks)]
    next_data = next_data[next_data["證券代碼"].isin(common_stocks)]

    train_data = train_data.sort_values("證券代碼")
    test_data = test_data.sort_values("證券代碼")
    next_data = next_data.sort_values("證券代碼")

    X_train = train_data[features].values
    y_train = train_data["y"].values
    X_test = test_data[features].values
    R_next = next_data["return"].values

    if np.any(np.isnan(X_train)) or np.any(np.isnan(X_test)) or np.any(np.isnan(R_next)):
        continue

    if len(np.unique(y_train)) < 2:
        continue

```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(penalty="l2", C=1/lambda_ridge, solver="lbfgs")
model.fit(X_train, y_train)

p_hat = model.predict_proba(X_test)[:, 1]

if len(p_hat) != len(R_next):
    continue

prop_positive = np.mean(y_train)
prop_p_hat = np.mean(p_hat > gamma)
quantiles = np.percentile(p_hat, [0, 5, 25, 50, 75, 95, 100])
R_ls = np.mean(np.sign(p_hat - gamma) * R_next)
R_lo = np.mean((p_hat > gamma).astype(int) * R_next)
R_b = np.mean(R_next)

results_lambda_5_gamma_04 = {
    "time": t,
    "prop_positive": prop_positive,
    "prop_p_hat": prop_p_hat,
    "quantiles": quantiles,
    "R_ls": R_ls,
    "R_lo": R_lo,
    "R_b": R_b
}

mv_ls_5_gamma_04 = np.mean([r["R_ls"] for r in results_lambda_5_gamma_04]) / np.std([r["R_ls"] for r in results_lambda_5_gamma_04]) if np.std([r["R_ls"] for r in results_lambda_5_gamma_04]) > 0 else 0
mv_lo_5_gamma_04 = np.mean([r["R_lo"] for r in results_lambda_5_gamma_04]) / np.std([r["R_lo"] for r in results_lambda_5_gamma_04]) if np.std([r["R_lo"] for r in results_lambda_5_gamma_04]) > 0 else 0
print(f"Question 4-3 (lambda=5,gamma=0.4): MV_LS={mv_ls_5_gamma_04:.4f}, MV_LO={mv_lo_5_gamma_04:.4f}")
```

✓ 0.4s Python

Question 4-3 (lambda=5,gamma=0.4): MV_LS=0.0943, MV_LO=0.1966

第三部分 用 LS 或 LO 算出來的結果依舊都是小於 benchmark 的結果